MPI_Allreduce() Implementation

Introduction

The **Message Passing Interface (MPI)** serves as a foundational framework for developing parallel applications in high-performance computing (HPC). Among its rich collection of collective communication routines, **MPI_Allreduce** stands out as a critical operation, enabling all processes within a communicator to contribute data and subsequently receive the globally reduced result. This operation is ubiquitous in scientific simulations, machine learning training, and numerical algorithms where global aggregations—such as sums, maxima, or minima—are required across all participating processes.

While the MPI standard defines the semantic behavior of MPI_Allreduce, it intentionally leaves the implementation details open to allow for optimization across diverse hardware architectures. Consequently, multiple algorithmic approaches have been developed to realize this collective operation, each with distinct performance characteristics that depend on factors like message size, number of processes, network topology, and system capabilities. Two classical and widely studied algorithms are: (1) the **Reduce-Broadcast** approach, which first performs a reduction to a single root process followed by a broadcast of the result, and (2) the **Ring Pipeline** algorithm, which organizes processes in a logical ring and pipelines data segments around the ring to progressively compute and disseminate the global reduction.

This project aims to implement, analyze, and compare these two fundamental MPI_Allreduce algorithms base on the **sum** operation. The primary **objectives** are:

1. To **develop** correct and efficient implementations of both the Reduce-Broadcast and Ring Pipeline algorithms using MPI point-to-point and collective operations.
2. To empirically **evaluate** and **compare** their performance (execution time) and scalability across a range of data sizes and process counts.
3. To understand the inherent **trade-offs** between the algorithms—such as latency, bandwidth utilization, memory overhead, and concurrency—and to identify the conditions under which each algorithm excels.

The implementation and testing framework developed for this project systematically measures the execution time of both algorithms for varying vector sizes (**from 10 to 10 million** 32-bit floating-point numbers) and for different numbers of MPI processes. Each algorithm is evaluated for correctness by comparing its output

against a known expected result. Performance metrics, including average execution time and speedup of Ring Pipeline relative to Reduce-Broadcast, are collected and analyzed. Additionally, a **theoretical memory usage analysis** is provided to highlight the memory footprint differences between the two approaches.

The remainder of this report details the design and implementation of both algorithms, describes the experimental setup and methodology, presents and discusses the performance results, and concludes with insights and potential directions for future optimization. Through this study, we gain practical experience in implementing non-trivial MPI communication patterns and deepen our understanding of how algorithmic choices impact the performance of collective operations in parallel computing.

Design

1. Reduce-Broadcast Algorithm

1.1 Design Approach

The Reduce-Broadcast approach implements MPI_Allreduce as two sequential collective operations:

1) **MPI_Reduce:** All processes send their local data to a designated root process (rank 0), where the reduction operation (MPI_SUM) is performed.

2) **MPI_Bcast:** The root process broadcasts the reduced result back to all processes.

This two-phase design leverages MPI's highly optimized collective routines and is straightforward to implement, although it may introduce a communication bottleneck at the root process.

1.2 Key Characteristics

- **Simplicity**: Leverages MPI's optimized collective operations.
- **Theoretical Complexity**: O(log P) communication steps, where P is the number of processes.
- **Memory Requirement**s: Each process maintains input and output buffers.
- **Communication Pattern**: Tree-based reduction followed by tree-based broadcast.
- **Network Utilization**: Root process becomes a bottleneck for large data sizes.

1.3 Implementation Considerations

The algorithm's simplicity is its primary strength, as it delegates optimization responsibility to the MPI implementation's internal collective algorithms. This approach typically yields good performance for moderate data sizes and process counts, though it may suffer from root process contention at scale.

2. Ring Pipeline Algorithm

The Ring Pipeline algorithm implements MPI_Allreduce through a segmented, point-to-point communication pattern organized in a logical ring topology. This approach aims to maximize bandwidth utilization and overlap computation with communication.

2.1 Pipeline Implementation

The pipeline implementation follows a sophisticated multi-phase strategy that ensures both correct reduction and efficient data distribution:

2.1.1 Dual-Phase Pipeline Architecture

The algorithm operates in two distinct phases executed sequentially for each batch of segments:

1) **Reduction Phase:** In the first phase, each process participates in a series of (P-1) communication steps where data segments flow clockwise around the ring (where P is the number of processes). During each step, processes exchange segments and perform local reductions, gradually accumulating the global sum across the ring.

2) **Distribution Phase:** Following reduction, a second pipeline phase propagates the fully reduced segments to all processes. This phase effectively broadcasts the results without requiring a dedicated broadcast operation, leveraging the same ring communication pattern.

2.1.2 Batch Processing Strategy

To maximize efficiency, the algorithm processes segments in batches where each batch contains exactly P segments (where P is the number of processes). This design ensures that:

- Each segment in the batch is initially owned by a different process.
- After (P-1) reduction steps, one process holds the fully reduced version of each segment.
- The distribution phase then propagates these reduced segments to all processes.

2.1.3 Edge Case Handling

The implementation carefully handles cases where the total number of segments is **not** an exact **multiple of P**. In these scenarios:

- The final batch contains fewer than P segments.
- Processes dynamically determine whether they need to send or receive based on segment availability.
- Special boundary checks prevent unnecessary communication and ensure correct termination.

2.1.4 Helper Functions for Segment Management

Two key helper functions manage the complex segment indexing:

1) **init_recv_seg:** This function initializes the receive segment index for each process at the start of each pipeline phase. The initial receive segment is strategically determined based on process rank and batch index, and it subsequently becomes the first send segment after processing, ensuring seamless pipeline flow between receive and send operations.

2) **update_send_recv_seg:** Updates segment indices after each communication step, calculating the next segment to send and receive while maintaining the circular ring traversal pattern.

These functions encapsulate the intricate index calculations, making the main algorithm logic more readable and maintainable while ensuring correct segment tracking throughout the pipeline.

2.2 Computation and Communication Overlap

The algorithm employs several techniques to maximize the overlap between computation and communication:

1) **Non-Blocking Operations:** All communication operations use MPI's non-blocking send and receive primitives (**MPI_Isend** and **MPI_Irecv**), allowing the program to continue execution while data transfers proceed.

2) **Asynchronous Progress:** The MPI implementation can make progress on communication operations in the background, potentially overlapping with both computational work and other communication operations.

This design minimizes idle time and ensures that both network bandwidth and CPU resources are utilized efficiently.

2.3 Segment Size Optimization

The performance of the ring pipeline algorithm is highly sensitive to the chosen segment size, which must balance pipeline startup overhead, memory bandwidth utilization, and network characteristics. **Two distinct strategies** for determining the segment size were implemented and evaluated:

1) **Heuristic Size-Based Thresholds:** The strategy employs a simple heuristic based on the total data size. For very small messages (≤ 64 KB), the entire data is treated as a single segment to avoid pipeline overhead. For medium-sized messages (≤ 4 MB), a fixed segment size of 64 KB is used, while larger messages use 256 KB segments. This approach provides a reasonable compromise without requiring runtime network measurements.

2) **Fixed-Size Segmentation:** As a baseline, a constant segment size of 65,536 elements (256 KB for single-precision floats) is also available. This simplistic method is useful for comparison and for environments where network characteristics are stable and predictable.

I chose the first approach because it can effectively balance pipeline overhead and data transmission efficiency in practice, and is a proven and reliable engineering compromise solution.

2.4 Design Advantages

1) **Bandwidth Efficiency:** The ring pattern utilizes all network links simultaneously, maximizing aggregate bandwidth.
2) **Load Balancing:** All processes perform equal amounts of communication and computation.
3) **Scalability:** The algorithm avoids centralized bottlenecks, enabling better scaling to large process counts.
4) **Memory Locality:** Segmented processing improves cache utilization and memory access patterns.

The Ring Pipeline algorithm represents a sophisticated approach to collective communication that addresses the limitations of tree-based methods through careful pipeline design, segment optimization, and computation-communication overlap.

Execution

```
cd The/Folder/of/The/Project
module load mpi/openmpi/4.1.4-gcc-4.8.5
mpicc main.c allreduce_reduce_bcast.c all_reduce_ring_pipeline.c -o main -lm
```

mpirun -np 10 ./main

Results and Analysis

1 Runtime Performance

1.1 Performance Evaluation Methodology

The primary objective of this evaluation was to empirically compare the execution time and scalability of the two custom MPI_Allreduce implementations (Reduce-Broadcast and Ring Pipeline) against each other and the native MPI_Allreduce routine.

1) Benchmark Design and Workload

To assess performance across a wide spectrum, we designed a comprehensive set of workloads varying in total data size (N).

Data Size (N): We tested 12 distinct data sizes, forming a geometric and linear progression: {10, 100, 1000, 10000, 100000, 1000000, 10000000, 100000000, 200000000, 300000000, 400000000, 500000000}. Each element is a single-precision floating-point number (float, 4 bytes). This range covers scenarios from tiny latency-bound messages (40 Bytes) to large bandwidth-bound messages (~1.86 GB).

Process Count (P): The tests were conducted with a fixed process count of P = 10, with processes distributed across multiple nodes to fully exercise the inter-node network.

2) Performance Metrics

The key metric extracted from the measurements is the **Average Execution Time**, reported in seconds for each algorithm (avg_time1 for Reduce-Broadcast, avg_time2 for Ring Pipeline, avg_time3 for native MPI_Allreduce).

To directly compare the efficiency of the two custom implementations, we calculated the Speedup of the Ring Pipeline over the Reduce-Broadcast approach using the formula: **Speedup** = Time(Reduce-Broadcast) / Time(Ring Pipeline).

A Speedup value greater than 1.0 indicates that the Ring Pipeline was faster for that specific workload, while a value less than 1.0 indicates the Reduce-Broadcast method was superior.

3) Measurement Procedure

To ensure statistical reliability and mitigate the impact of transient system noise, each specific configuration, which is defined by the algorithm (Reduce-Broadcast, Ring Pipeline, or native MPI_Allreduce) and the data size, was executed for **10 consecutive iterations**. The raw execution time for each iteration was recorded. The reported performance metric for that specific configuration is the arithmetic **mean** of these 10 individual time measurements.

1.2 Results

P = 10 (Image 1 and Image 2) :

Test Size (float)	Time (Reduce+Broadcast)	Time (Ring Pipeline)	Time (MPI_allreduce)	Speedup (Ring vs Reduce+Broadcast)
10	0.000058 s	0.0000020 s	0.000011 s	2.94
100	0.000027 s	0.000036 s	0.000017 s	0.74
1,000	0.000057 s	0.000123 s	0.000040 s	0.47
10,000	0.000116 s	0.000430 s	0.000122 s	0.27
100,000	0.001262 s	0.001224 s	0.001122 s	1.03
1,000,000	0.011850 s	0.009606 s	0.011283 s	1.23
10,000,000	0.102186 s	0.089393 s	0.078491 s	1.14
100,000,000	1.028346 s	0.980764 s	0.717852 s	1.05
500,000,000	6.163698 s	4.832830 s	6.466779 s	1.28

```
=== Testing with data size: 10 , process number: 10 ===
Test size                        : 10
Result                           : OK
Reduce+Broadcast: Time           = 0.000058 s
Ring Pipeline: Time              = 0.000020 s
MPI_allreduce: Time              = 0.000011 s
Speedup (Ring vs Reduce+Broadcast): 2.94

=== Testing with data size: 100 , process number: 10 ===
Test size                        : 100
Result                           : OK
Reduce+Broadcast: Time           = 0.000027 s
Ring Pipeline: Time              = 0.000036 s
MPI_allreduce: Time              = 0.000017 s
Speedup (Ring vs Reduce+Broadcast): 0.74

=== Testing with data size: 1,000 , process number: 10 ===
Test size                        : 1,000
Result                           : OK
Reduce+Broadcast: Time           = 0.000057 s
Ring Pipeline: Time              = 0.000123 s
MPI_allreduce: Time              = 0.000040 s
Speedup (Ring vs Reduce+Broadcast): 0.47

=== Testing with data size: 10,000 , process number: 10 ===
Test size                        : 10,000
Result                           : OK
Reduce+Broadcast: Time           = 0.000116 s
Ring Pipeline: Time              = 0.000430 s
MPI_allreduce: Time              = 0.000122 s
Speedup (Ring vs Reduce+Broadcast): 0.27

=== Testing with data size: 100,000 , process number: 10 ===
Test size                        : 100,000
Result                           : OK
Reduce+Broadcast: Time           = 0.001262 s
Ring Pipeline: Time              = 0.001224 s
MPI_allreduce: Time              = 0.001122 s
Speedup (Ring vs Reduce+Broadcast): 1.03

=== Testing with data size: 1,000,000 , process number: 10 ===
Test size                        : 1,000,000
Result                           : OK
Reduce+Broadcast: Time           = 0.011850 s
Ring Pipeline: Time              = 0.009606 s
MPI_allreduce: Time              = 0.011283 s
Speedup (Ring vs Reduce+Broadcast): 1.23

=== Testing with data size: 10,000,000 , process number: 10 ===
Test size                        : 10,000,000
Result                           : OK
Reduce+Broadcast: Time           = 0.102186 s
Ring Pipeline: Time              = 0.089393 s
MPI_allreduce: Time              = 0.078491 s
Speedup (Ring vs Reduce+Broadcast): 1.14
```

Image 1: Speedup (size from 10 - 10,000,000)

```
=== Testing with data size: 100,000,000 , process number: 10 ===
Test size                        : 100,000,000
Result                           : OK
Reduce+Broadcast: Time           = 1.028346 s
Ring Pipeline: Time              = 0.980764 s
MPI_allreduce: Time              = 0.717852 s
Speedup (Ring vs Reduce+Broadcast): 1.05

=== Testing with data size: 200,000,000 , process number: 10 ===
Test size                        : 200,000,000
Result                           : OK
Reduce+Broadcast: Time           = 1.874550 s
Ring Pipeline: Time              = 1.813511 s
MPI_allreduce: Time              = 1.436029 s
Speedup (Ring vs Reduce+Broadcast): 1.03

=== Testing with data size: 300,000,000 , process number: 10 ===
Test size                        : 300,000,000
Result                           : OK
Reduce+Broadcast: Time           = 2.853434 s
Ring Pipeline: Time              = 2.845465 s
MPI_allreduce: Time              = 2.638202 s
Speedup (Ring vs Reduce+Broadcast): 1.00

=== Testing with data size: 400,000,000 , process number: 10 ===
Test size                        : 400,000,000
Result                           : OK
Reduce+Broadcast: Time           = 3.927018 s
Ring Pipeline: Time              = 3.721920 s
MPI_allreduce: Time              = 4.341227 s
Speedup (Ring vs Reduce+Broadcast): 1.06

=== Testing with data size: 500,000,000 , process number: 10 ===
Test size                        : 500,000,000
Result                           : OK
Reduce+Broadcast: Time           = 6.163698 s
Ring Pipeline: Time              = 4.832830 s
MPI_allreduce: Time              = 6.466779 s
Speedup (Ring vs Reduce+Broadcast): 1.28
```

Image 2: Speedup (size from 100,000,000 - 500,000,000)

1.3 Analysis

1.3.1 Overall Trend and Performance Regimes

The Speedup (Ring vs Reduce-Broadcast) data defines three distinct performance regimes:

1) **Tiny Messages (10 elements):** Ring Pipeline is significantly faster (2.94x speedup).
2) **Small to Medium Messages (100 to 10,000 elements):** Reduce-Broadcast becomes dominant, with its advantage peaking at 10,000 elements (3.70x speedup for Reduce-Broadcast, or 0.27x for Ring).
3) **Large Messages (100,000 to 500,000,000 elements):** Ring Pipeline regains and maintains a consistent, moderate performance advantage, with speedups between 1.03x and 1.28x.

1.3.2 Analysis of Algorithmic Behavior

The underlying reasons for this trend are rooted in the fundamental design of each algorithm:

For Tiny Messages (10 elements): The overwhelming advantage of the **Ring Pipeline** is due to its **lower latency overhead**. The **Reduce-Broadcast** approach, which uses MPI's built-in MPI_Reduce and MPI_Bcast, incurs the **fixed startup**

costs of two separate collective operations, even for a single float. The Ring algorithm's point-to-point, single-pipeline communication pattern has a simpler and faster startup in this microscopic regime.

For Small to Medium Messages (100 - 10,000 elements): This is the domain where the logarithmic vs. linear scaling of communication steps becomes critical. The **Reduce-Broadcast** algorithm's tree-based pattern completes in **O(log P)** steps. In contrast, the **Ring Pipeline** requires **O(P)** steps (9 steps for P=10). For these message sizes, the per-step latency and management overhead of the Ring's 9 sequential steps **outweigh** its **bandwidth advantages**, causing significant performance degradation. The minimum speedup at 10,000 elements likely represents the point where the Ring's pipeline is at its highest overhead relative to the data transfer time.

For Large Messages (≥ 100,000 elements): As message size grows, **bandwidth utilization** becomes the dominant factor over latency. The Ring Pipeline excels here because it creates a sustained, balanced flow of data around the ring, efficiently saturating the network links. The **Reduce-Broadcast** method, while having fewer steps, can create **bottlenecks at the root process** during both the reduce and broadcast phases, limiting its effective aggregate bandwidth. The **Ring's** ability to **overlap communication with computation** in its pipeline also starts to yield benefits for these larger data chunks.

1.3.3 The data suggests divergent scalability properties:

- The **Reduce-Broadcast** algorithm shows stronger latency scalability for **small** messages, as its performance disadvantage is minimal and only appears for very tiny data.
- The **Ring Pipeline** algorithm demonstrates better bandwidth scalability for **large** messages, maintaining a stable ~1.2x advantage as the data size scales by over four orders of magnitude (from 100,000 to 500,000,000 elements). This indicates its performance scales more favorably with increasing data volume.

2 Memory Efficiency and Scalability

2.1 Performance Evaluation Methodology

The assessment of memory efficiency was conducted through a **theoretical memory footprint analysis** rather than runtime measurement. This approach isolates the inherent, algorithm-specific memory requirements, providing a clear comparison independent of system-specific memory allocator behavior or runtime state.

The analysis calculates the **peak memory footprint per process** for each algorithm, which is the maximum amount of memory a single process must allocate

simultaneously to execute the operation. The footprint is derived from the algorithm's design and the problem parameters: total data count (*count*), datatype size (*type_size*), and for the Ring Pipeline, the chosen segment size (*segment_size*).

The analysis for each algorithm is structured as follows:

1. Reduce-Broadcast Algorithm:

- **Input Buffer:** A buffer of size *count * type_size bytes* to hold the local data contributed to the reduction.
- **Output Buffer:** A buffer of size *count * type_size bytes* to store the final, globally reduced result received from the broadcast.
- **MPI Internal Overhead:** An estimated, fixed overhead (conservatively **estimated at 0.1 MB**) to account for potential internal buffering within the MPI library during the collective operations. The total per-process footprint is the sum of these three components.

2. Ring Pipeline Algorithm:

- **Primary Buffers:** An input and an output buffer, identical to those in the Reduce-Broadcast method, each of size count * type_size bytes.
- **Pipeline Buffers:** Two additional buffers are the key differentiator:
  - **Send Buffer:** Holds one segment of data to be sent to the next process in the ring.
  - **Recv Buffer:** Holds one segment of data received from the previous process in the ring. The size of each is *segment_size * type_size* bytes. The segment size is determined by the heuristic strategy described in the design section.
- **Total Footprint:** The sum of the two primary buffers and the two pipeline buffers.

Comparison Metric: The results from the two models are directly compared. The **absolute difference** in memory usage *(ring_mem - reduce_bcast_mem)* and the **relative percentage** difference *((difference / reduce_bcast_mem) * 100%)* are calculated. A positive percentage indicates the Ring Pipeline has a higher memory footprint.

2.2 Results

P = 10 (Image 3 and Image 4) :

Test Size	Memory usage per process (Reduce + Broadcast)	Memory usage per process (Ring Pipeline)	Ring uses LESS memory by
10	0.10 MB	0.00 MB	99.8%
100	0.10 MB	0.00 MB	98.5%
1,000	0.11 MB	0.02 MB	85.8%
10,000	0.18 MB	0.14 MB	21.3%
100,000	0,86 MB	0.83 MB	4.3%
1,000,000	7,73 MB	7.88 MB	-1.9%
10,000,000	76.39 MB	76.54 MB	-0.2%
100,000,000	763.04 MB	763.19 MB	-0.0%

```
=== Theoretical Memory Analysis ===

test size = 10,000

process number = 10

1. Reduce + Broadcast:
   Input buffer:            0.04 MB
   Output buffer:           0.04 MB
   MPI internal:            0.10 MB (estimated)
   Total per process:       0.18 MB

2. Ring Pipeline:
   Input buffer:            0.04 MB
   Output buffer:           0.04 MB
   Segment size:           8192 elements
   Total segments:           2
   Send buffer:             0.03 MB
   Recv buffer:             0.03 MB
   Total buffers:           0.06 MB
   Total per process:       0.14 MB

3. Comparison:
   Reduce+Bcast:            0.18 MB
   Ring Pipeline:           0.14 MB
   Difference:             -0.04 MB (-21.3%)
   Ring uses LESS memory by 21.3%

==================================
```

Image 3: Memory Analysis (size= 10,000)

```
=== Theoretical Memory Analysis ===

test size = 100,000,000

process number = 10

1. Reduce + Broadcast:
   Input buffer:          381.47 MB
   Output buffer:         381.47 MB
   MPI internal:            0.10 MB (estimated)
   Total per process:     763.04 MB

2. Ring Pipeline:
   Input buffer:          381.47 MB
   Output buffer:         381.47 MB
   Segment size:           32768 elements
   Total segments:           3052
   Send buffer:             0.12 MB
   Recv buffer:             0.12 MB
   Total buffers:           0.25 MB
   Total per process:     763.19 MB

3. Comparison:
   Reduce+Bcast:          763.04 MB
   Ring Pipeline:         763.19 MB
   Difference:              0.15 MB (+0.0%)
   Ring uses MORE memory by 0.0%

==================================
```

Image 4: Memory Analysis (size= 100,000,000)

2.2 Analysis

2.2.1 Analysis of Memory Footprint Trends

The results, detailed for a configuration of P=10 processes, reveal a nuanced relationship between memory overhead and problem size, defined by three distinct regimes based on the total data size (N) :

1) **For very small data sizes (10 to 1,000 elements):** The **Ring Pipeline** algorithm demonstrates a **dramatic memory advantage**, using between 85.8% to 99.8% less memory than the Reduce-Broadcast method. This is because, at this microscopic scale, the optimally chosen *segment_size* is often **equal to or larger than N** itself. Consequently, the algorithm's pipeline buffers (*send_seg* and *recv_seg*) can function as the primary working storage, potentially avoiding the allocation of separate, full-sized input and output buffers. In contrast, the Reduce-

Broadcast method incurs a **fixed overhead for MPI internal operations**, which dominates its total memory footprint when `N` is very small, making it comparatively inefficient.

2) **As the data size grows to a medium scale (10,000 to 100,000 elements):** The memory advantage of the **Ring Pipeline diminishes rapidly**, from 21.3% down to just 4.3%. In this regime, the ***O(N)*** term, which is the memory required for the primary input and output buffers, begins to dominate the total footprint for both algorithms. The **fixed overheads** (MPI's internal buffers for Reduce-Broadcast and the pipeline buffers for Ring) become progressively **less significant** relative to the linearly growing main buffers. The algorithms' memory consumption thus starts to converge.

3) **For large-scale problems (1,000,000 elements and beyond):** the difference in memory footprint becomes **negligible**, ranging from -1.9% to -0.0%. Here, the ***O(N)*** term **completely dominates** the memory cost. The Ring Pipeline's additive overhead of *2 * segment_size* and the Reduce-Broadcast's MPI internal overhead are both constant and insignificant compared to the massive, linearly-scaled primary buffers. Consequently, the per-process memory footprint of both algorithms converges and becomes virtually identical, governed by the same *O(N)* asymptotic complexity.

2.2.2 Conclusion on Memory Scalability

The analysis conclusively shows that for **large-scale data**, both algorithms exhibit **equivalent linear memory scalability, O(N)**. The minor, near-constant difference in their memory footprints becomes irrelevant as *N* grows. Therefore, while the Ring Pipeline can offer significant memory savings for trivial problem sizes, memory efficiency is not a differentiating factor for practical, large-scale computations. The choice between the two algorithms should be primarily informed by their runtime performance characteristics, which, as established in Section 1, show a strong and meaningful dependency on the message size.

Summary

This report presents a comprehensive study implementing and comparing two core algorithms for the **MPI_Allreduce** operation: the **Reduce-Broadcast** method and the **Ring Pipeline** algorithm. The project moved beyond theoretical understanding to practical implementation and empirical evaluation on a multi-node cluster.

Our performance analysis revealed a clear **trade-off** governed by message size. The **Ring Pipeline** algorithm excelled with very **small** messages due to lower latency and with very large messages due to superior bandwidth utilization. Conversely, the **Reduce-Broadcast** method was more efficient for a range of **small-to-medium** messages, benefiting from its logarithmic communication steps. This creates a critical

performance crossover point, demonstrating that no single algorithm is optimal for all scenarios, the choice must be data-driven.

Regarding memory, theoretical analysis showed that while the Ring Pipeline can be significantly more efficient for trivial data sizes, both algorithms exhibit **asymptotically equivalent linear scalability** for **large-scale** problems. Therefore, b, not memory footprint, is the **decisive** factor for algorithm selection.

This project underscored that parallel algorithm performance is **non-intuitive** and **highly context-dependent**. It highlighted the trade-off between latency and bandwidth, the impact of communication patterns, and the necessity of empirical validation over purely theoretical models. The experience reinforced that designing efficient parallel software requires careful consideration of both algorithmic properties and the specific characteristics of the target workload and hardware.over purely theoretical models. The experience reinforced that designing efficient parallel software requires careful consideration of both algorithmic properties and the specific characteristics of the target workload and hardware.